iam-jit · GitHub JIT tokens

# Scoped, short-lived GitHub tokens

What it looks like to issue and use a least-privilege GitHub token — once as a **person** clicking through the web UI, and once as an **agent** scoping itself over MCP with no human in the loop.

**Why this exists.** A malicious skill held a broad, standing GitHub token and infected every PR of every repo it could reach. iam-jit replaces that with a token scoped to the **fewest repos × one permission × ≤1 hour**, instantly revocable — so an infected machine or agent has a far smaller blast radius.

**Part 1** — For a person: the `/github` web UI · **Part 2** — For an agent: the `github_scope_self_for_task` MCP tool · **Part 3** — What the token can & can't do (granularity)

## Part 1 — For a person (the web UI)

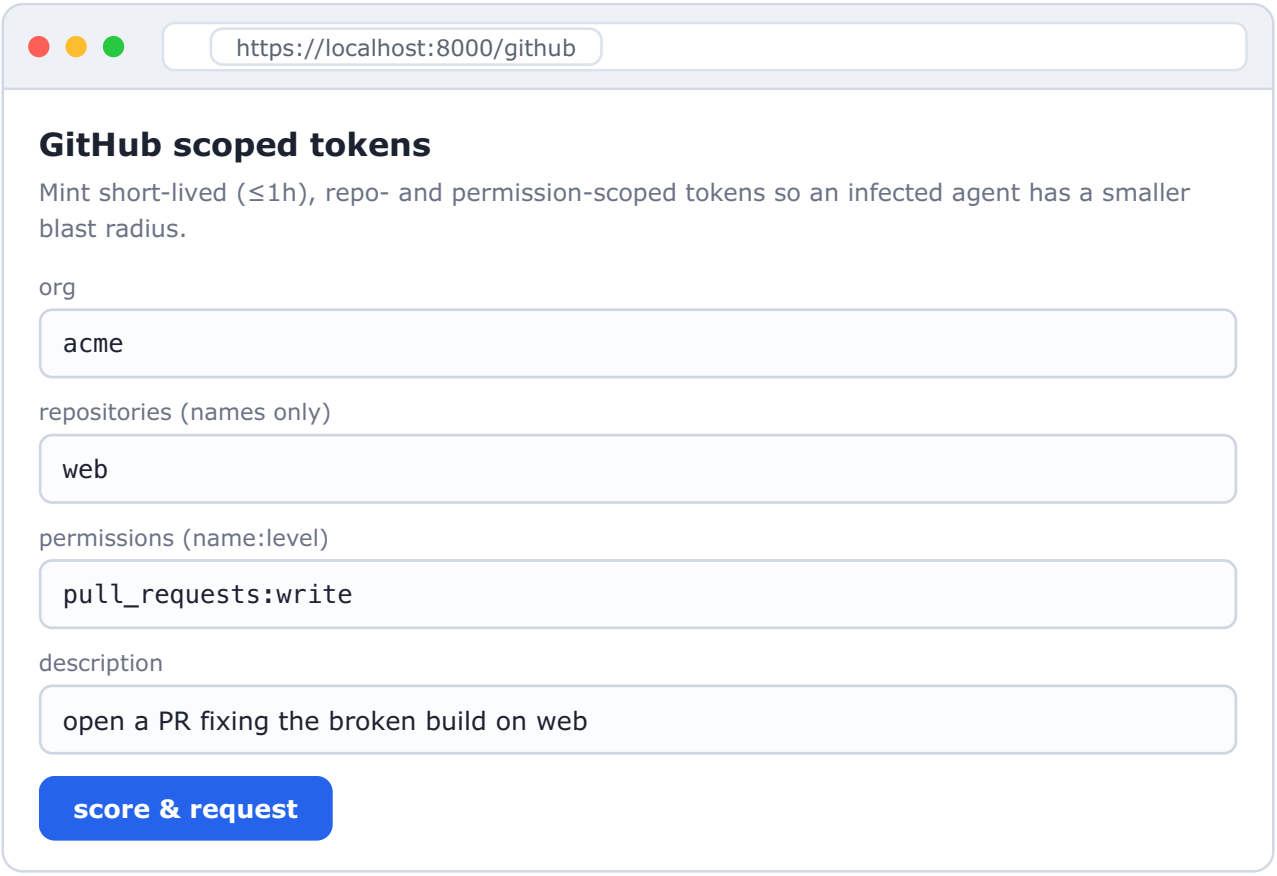
A developer opens iam-jit `serve` in the browser and asks for exactly the access a task needs. Low-risk scopes issue instantly; broad or high-impact ones wait for an admin.

### 1 One-time: an admin connects the GitHub org

A GitHub App (the issuer) is registered once. After that, iam-jit can mint scoped tokens for any connected org.

```
$ iam-jit github connect \
    --org acme \
    --app-id 123456 \
    --installation-id 98765432 \
    --private-key-path ~/.iam-jit/acme.private-key.pem
✓ connected GitHub org 'acme' (installation 98765432)
```

### 2 Request a scoped token

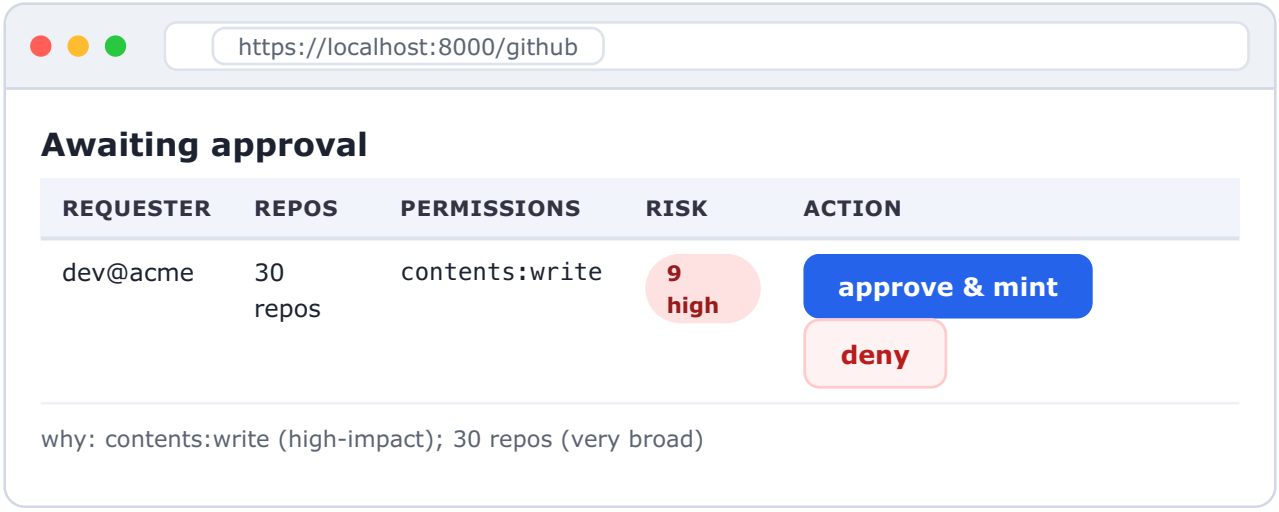


## 3 Low risk → issued instantly, shown once

One repo + `pull_requests:write` scores **low** → auto-approved. The token is displayed a single time.

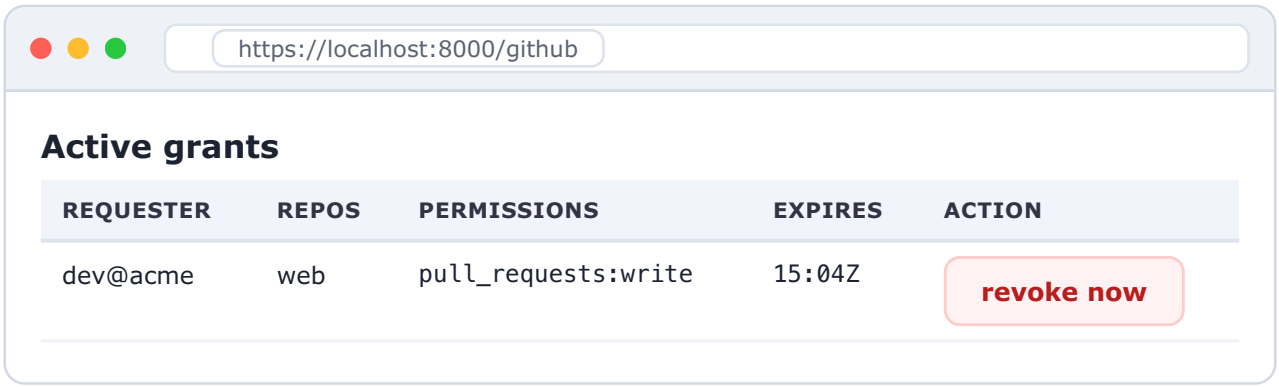

## 4 High risk → queued for an admin (mints nothing)

A request for `contents:write` across 30 repos scores **high**. **No token is minted** — it waits in the approver queue. The high-risk request never reaches GitHub until a human says yes.



## 5 Active grants → revoke early

Every issued token is listed with its expiry. An admin can kill one instantly (`DELETE /installation/token`) without waiting for the TTL.



## Part 2 — For an agent (MCP, no human)

An AI agent scopes *itself*: it calls one MCP tool with the repos + permissions its task needs. Low-risk → it gets a token back and works. High-risk → it gets `needs_approval` and **no token** — the blast-radius cap holds even for a compromised agent.

### The agent asks for exactly what the task needs

```
// MCP tool call
github_scope_self_for_task({
  org: "acme",
  description: "open a PR fixing the broken build on web",
  repositories: ["web"],
  permissions: { pull_requests: "write" }
})
```

### Low risk → token returned, agent proceeds

```
// response
{
  decision: "issued",
  risk_score: 3, band: "low",
  repositories: ["web"],
  permissions: { pull_requests: "write" },
  token: "ghs_9Hb2…Qx7r",
  expires_at: "2026-06-06T15:04:00Z"
}
```

### High risk → no token; the agent is told to get a human

```
github_scope_self_for_task({ org: "acme",
  repositories: ["…30 repos…"], permissions: { contents: "write" } })

// response — NOTHING was minted
{
  decision: "needs_approval",
  risk_score: 9, band: "high",
  risk_factors: ["contents:write (high-impact)", "30 repos (very broad)"]
  // no token field — a compromised agent cannot self-issue broad access
}
```

### The agent uses the token — and GitHub enforces the scope

The same blast-radius matrix the UAT proves. Every ❌ is GitHub rejecting server-side, not iam-jit asking nicely.

```
// token scoped to: web / pull_requests:write
→ open a PR on web ............................. 200 OK  ✅ in scope
→ push contents to web (ungranted perm) ....... 403     ❌ denied
→ read repo api (out of scope) ................ 404     ❌ invisible
→ any call after ≤1h TTL ...................... 401     ❌ expired
→ any call after revoke ....................... 401     ❌ killed
```

> A fully compromised agent can only do what the token allows — one repo, one permission, for under an hour, revocable at any moment. The token it never managed to over-scope is the whole point.

# Part 3 — What the token can & can't do

A mint has exactly three knobs. Knowing the ceiling matters for honest threat-modeling.

## Three axes of granularity

| AXIS | HOW FINE | FLOOR |
|---|---|---|
| **Repositories** | Per-repo, by name. Out-of-scope repo → 404 (can't even confirm it exists). Empty list is refused (GitHub reads it as *all* repos). | One repo |
| **Permissions** | ~40 categories, each `read`/`write`/`admin`, independent. Must be a subset of the App's ceiling. | One category, read |
| **Time** | Fixed ≤1h TTL (GitHub-set). Instant `DELETE /installation/token` revoke. | Seconds (via revoke) |

## Common permission categories

contents · pull_requests · issues · actions · workflows · secrets · administration · checks · statuses · deployments · packages · environments · metadata (always read)

## The hard ceiling — what you CANNOT make finer

| YOU MIGHT WANT… | REALITY |
|---|---|
| "write only to `src/`" | **No path/file scoping.** `contents:write` = any file, any branch in the in-scope repos. |
| "only push to `feature/*`" | **No branch scoping** in the token (branch *protection rules* are a separate repo setting). |
| "only the *create-PR* endpoint" | **No per-endpoint scoping** — `pull_requests:write` is all PR-write endpoints. |
| "just this one secret" | **No sub-resource scoping** — it's the whole `secrets` category for the in-scope repos. |

> **Net:** the realistic minimum blast radius is **one repository × one permission category × ≤1 hour, instantly revocable**. Within a single repo a `write` grant is repo-wide — which is exactly why iam-jit's scorer leans on fewest-repos + read-by-default + scoring write/admin high + short TTL, rather than pretending a permission can be sub-divided.